```
~sorter()                                    <-- ❹
{
    end_of_data=true;                        <-- ❺

    for(unsigned i=0;i<threads.size();++i)
    {
        threads[i].join();                   <-- ❻
    }
}

void try_sort_chunk()
{
    boost::shared_ptr<chunk_to_sort > chunk=chunks.pop();     <-- ❼
    if(chunk)
    {
        sort_chunk(chunk);                   <-- ❽
    }
}
```

Here, the `parallel_quick_sort` function ⓳ delegates most of the functionality to the `sorter` class ❶, which provides an easy way of grouping the stack of unsorted chunks ❷ and the set of threads ❸. The main work is done in the `do_sort` member function ❾, which does the usual partitioning of the data ❿. This time, rather than spawning a new thread for one chunk, it pushes it onto the stack ⓫ and spawns a new thread while you still have processors to spare ⓬. Because the lower chunk might be handled by another thread, you then have to wait for it to be ready ⓭. In order to help things along (in case you're the only thread or all the others are already busy), you try to process chunks from the stack on this thread while you're waiting ⓮. `try_sort_chunk` just pops a chunk off the stack ❼ and sorts it ❽, storing the result in the `promise`, ready to be picked up by the thread that posted the chunk on the stack ⓯.

Your freshly spawned threads sit in a loop trying to sort chunks off the stack ⓱ while the `end_of_data` flag isn't set ⓰. In between checking, they yield to other threads ⓲ to give them a chance to put some more work on the stack. This code relies on the destructor of your `sorter` class ❹ to tidy up these threads. When all the data has been sorted, `do_sort` will return (even though the worker threads are still running), so your main thread will return from `parallel_quick_sort` ⓴ and thus destroy